Formalizing Local Memory Hygiene Contracts for GPU Confidential Computing: A Compiler-Enforced and Attestation-Bound Framework

Tripti Sharma, Arshad Ali, Guruansh Singh
Vivekananda Institute of Professional Studies – Technical Campus (VIPS-TC),
Guru Gobind Singh Indraprastha University (GGSIPU), Delhi, India

***Abstract*—*Graphics Processing Units (GPUs) are the computational substrate for large language model (LLM) inference, increasingly in multi-tenant cloud settings. The LeftoverLocals vulnerability (CVE-2023-4969) showed that several GPU vendors fail to clear on-chip local (shared) memory between kernel executions, letting a co-resident adversary recover another tenant's LLM prompts and responses. Emerging GPU Confidential Computing (GPU-CC) architectures, such as NVIDIA Hopper H100 and AMD SEV-SNP-backed designs, scrub device memory at virtual-machine (VM) reassignment boundaries, but offer no formal, verifiable guarantee of per-kernel local memory hygiene for intra-VM, multi-kernel workloads. We argue this fine-grained hygiene is an unaddressed, security-critical gap. This paper makes four contributions. First, we formally define the Local Memory Hygiene Contract (LMHC): a vendor-agnostic policy specifying sensitive local state, a mandatory zeroization invariant at kernel exit, a threat model, and measurable compliance criteria. Second, we present a compiler-level source-to-source transformation that identifies sensitive buffers via taint analysis and inserts optimization-resistant scrubbing at all kernel exit points. Third, we propose an attestation binding that extends GPU-CC reports with an LMHC compliance bit and a kernel measurement hash. Fourth, we provide a proof-of-concept demonstration and a theoretical overhead analysis indicating that amortized scrubbing cost is modest. We position this work as a formalization foundation for verifiable GPU local memory hygiene.*

***Index Terms*—*GPU confidential computing, trusted execution environment, memory hygiene, side-channel, LeftoverLocals, compiler enforcement, remote attestation, LLM security.*

1. Introduction

Graphics Processing Units (GPUs) have become the dominant hardware platform for training and serving machine learning models, and in particular for large language model (LLM) inference. As LLM services are consolidated onto shared accelerators in public clouds, a single physical GPU is frequently multiplexed across many users. In this setting, the prompts a user submits and the responses a model generates are sensitive: they may contain proprietary instructions, personal data, or regulated content. Ensuring that one tenant cannot observe another tenant's data is therefore a first-order security requirement, not merely a quality-of-service concern.

The LeftoverLocals vulnerability, disclosed in January 2024 and tracked as CVE-2023-4969, made this threat concrete [1], [2]. LeftoverLocals exploits the fact that, on several GPU architectures, the on-chip local memory region (a software-managed scratchpad) is not cleared between successive kernel invocations. A malicious "listener" kernel can allocate local memory and read the residual contents left by a previously executed kernel belonging to another process or container. The original authors demonstrated that the attack can be implemented using a GPU kernel consisting of fewer than ten lines of code [1]. an attacker could reconstruct another user's interactive LLM session (e.g., a `llama.cpp` session), because matrix-multiplication routines stage intermediate values—including token data—through local memory. The vulnerability was confirmed when LeftoverLocals was reported to affect GPUs from Apple, AMD, Qualcomm, and Imagination Technologies, while testing conducted during coordinated disclosure did not identify the vulnerability on NVIDIA, Intel, or Arm GPUs. [1] and the vulnerability was assigned CVE-2023-4969 and received a CVSS v3.1 base score of 6.5 (Medium), reflecting its ability to expose sensitive information through GPU local-memory leakage [2]. The root cause is structural: no enforced zeroization step exists at kernel boundaries, and vendor remediations were issued as advisory source-code changes rather than as a verifiable contract.

In parallel, the industry has introduced GPU Confidential Computing (GPU-CC) to protect data in use on the accelerator. NVIDIA's Hopper H100 extends a CPU-side Trusted Execution Environment (TEE)—realized with

AMD SEV-SNP, Intel TDX, or Arm CCA—to the GPU: data crossing the PCIe bus is encrypted through bounce buffers, the GPU exposes a hardware root of trust and remote attestation, and a compute-protected region (CPR) of device memory is isolated from the untrusted hypervisor [3], [4]. AMD SEV-SNP provides the confidential VM (CVM) abstraction on the CPU, adding memory-integrity protection via a Reverse Map Table and hardware-rooted attestation [5]. NVIDIA Hopper Confidential Computing documentation describes memory and state scrubbing during GPU reset and Physical Function Level Reset (PF-FLR) events that occur as part of confidential-computing lifecycle management and tenant reassignment. However, publicly available documentation does not explicitly describe per-kernel local-memory zeroization guarantees [3].

Herein lies the gap that motivates this paper. VM-granularity scrubbing protects against a new VM inheriting a prior VM's memory, but it says nothing about hygiene *within* a confidential VM across multiple kernel launches. Real LLM serving stacks dispatch long pipelines of kernels, and modern GPUs support intra-VM multiplexing through mechanisms such as the Multi-Process Service (MPS) and Multi-Instance GPU (MIG) [CITE: add references for MPS/MIG intra-GPU sharing and any isolation analyses]. In these regimes, the LeftoverLocals class of leakage—residual sensitive data persisting in local memory between kernels—remains possible inside the protected boundary, orthogonal to global-memory encryption. We observe four concrete deficiencies: (i) no formalized per-kernel local memory hygiene policy exists for intra-VM, multi-kernel confidential workloads; (ii) vendors issue "clear memory" as advisory guidance without a contract that can be verified or attested; (iii) production compiler toolchains (NVCC, ROCm) do not automatically enforce local-buffer scrubbing before kernel exit; and (iv) GPU-CC attestation quotes contain no hygiene capability bit or measurement indicating whether inter-kernel local-memory zeroization is active.

Contributions. Addressing these deficiencies, this paper contributes:

(1) A formal definition of the Local Memory Hygiene Contract (LMHC). A precise, vendor-agnostic policy specifying which regions constitute sensitive local state, the mandatory zeroization invariant before kernel exit, the threat-model scope (honest-but-curious co-tenant and compromised driver), and measurable compliance criteria (Section 4).

(2) A compiler-level enforcement mechanism. A source-to-source transformation that uses taint analysis to identify sensitive buffers and inserts scrubbing loops at all kernel exit points, with dead-store-elimination awareness to prevent the compiler from removing the scrubbing code (Section 5).

(3) An attestation binding. An extension to the GPU-CC attestation report that includes an LMHC compliance bit and a kernel measurement hash, letting a remote verifier confirm hygiene enforcement before sharing sensitive data with a GPU workload (Section 6).

(4) A proof-of-concept and overhead analysis. A conceptual demonstration of the leakage pattern and its remediation, plus a theoretical overhead model for representative kernel classes, arguing that amortized zeroization cost is acceptable (Section 7).

Organization. Section 2 reviews the GPU memory hierarchy, the LeftoverLocals attack, GPU-CC architectures, and existing scrubbing approaches. Section 3 presents the threat model and formalizes the LMHC. Section 4 (numbered as the enforcement section below) develops the compiler transformation, and Section 5 describes the attestation binding. Section 6 reports the proof-of-concept evaluation and overhead estimation. Section 7 discusses limitations and generalizability, and Section 8 concludes. We position this work as foundational formalization rather than a production implementation.

2. Background and Related Work

2.1 GPU Memory Hierarchy

A GPU exposes a tiered memory hierarchy. Per-thread *registers* are fastest and private to a thread. An on-chip, software-managed scratchpad—called *shared memory* in CUDA and *local memory* in OpenCL—is shared by all threads of a cooperative block (work-group) and is used to stage data for reuse and inter-thread communication. Off-chip *global memory* (device DRAM) is large but high-latency. A terminology caution is warranted: CUDA's term "local memory" denotes per-thread spill storage that physically resides in global memory, whereas the on-chip scratchpad CUDA calls "shared" corresponds to OpenCL "local.", OpenCL local memory corresponds

conceptually to CUDA shared memory, both referring to an on-chip software-managed scratchpad shared among work-items (threads) within a work-group or thread block. To maintain consistency with the LeftoverLocals literature, we use the term *local memory* to denote this scratchpad throughout the paper [18][19]. The capacity of on-chip local/shared memory is architecture dependent and typically ranges from tens to hundreds of kilobytes per streaming multiprocessor; for example, NVIDIA Hopper H100 supports up to 228 KB of shared memory per SM [3], which bounds the volume of recoverable residue.

2.2 LeftoverLocals: Attack Mechanism and Impact

LeftoverLocals is a residue-recovery attack [1]. On affected platforms, when a kernel finishes, the local memory it used is not zeroed before the region is reused by a subsequent kernel—potentially from a different process, container, or tenant. An attacker dispatches a listener kernel that allocates local memory and copies its (uninitialized) contents to global memory for inspection. Because performance-critical routines such as general matrix multiply (GEMM) tile their operands through local memory, fragments of a victim's LLM activations and tokens are present in the residue, enabling cross-process reconstruction of model outputs [1]. The vendor-recommended mitigation is to clear local memory in kernel source code; however, Implementations that perform local-memory clearing through a separate wipe kernel remain vulnerable because a malicious listener kernel may be scheduled between the computation kernel and the wipe operation, allowing recovery of residual data [1]. This observation directly motivates an enforced, in-kernel zeroization invariant rather than best-effort external scrubbing.

2.3 GPU Confidential Computing Architectures

GPU-CC extends data-in-use protection to the accelerator. On NVIDIA Hopper H100, enabling confidential mode partitions device memory into protected and unprotected regions, encrypts CPU–GPU transfers via bounce buffers, and gates the GPU behind a hardware root of trust [3], [4]. Attestation measures the GPU firmware and configuration, signs the measurement with a device certificate chain, and lets a verifier allows a verifier to compare attestation evidence against reference measurements contained in a Trusted Computing Group (TCG) Reference Integrity Manifest (RIM), thereby determining whether the device software and configuration match expected values. The transport for device measurements follows the DMTF Security Protocol and Data Model (SPDM), a protocol designed for secure component authentication and attestation that has been subjected to formal security analysis [3],[9],[22]. On the CPU side, AMD SEV-SNP creates CVMs whose private pages are encrypted and integrity-protected through the Reverse Map Table, with attestation reports issued by the AMD Secure Processor [5]; Intel TDX provides an analogous trust-domain abstraction. Work on trusted device I/O (AMD SEV-TIO and the PCI-SIG TDISP specification) brings accelerators into the guest trust boundary over an authenticated PCIe fabric [8]. Earlier academic systems, notably Graviton [20] and HIX [21], established the feasibility of GPU TEEs prior to commercial support. Recent surveys and benchmark studies characterize the architecture and performance of these systems [14], [15].

2.4 Existing Memory-Scrubbing Approaches and Their Gaps

Two scrubbing regimes are relevant. First, GPU-CC performs a coarse device-memory scrub at VM reassignment (PF-FLR), engaging a memory lock that blocks access until the wipe completes, which also mitigates cold-boot attacks [3], [4]. This is necessary but operates at the wrong granularity for inter-kernel hygiene. Second, on the CPU, secure memory-wipe primitives such as `memset_s` and `explicit_bzero` exist precisely because a naive `memset` that clears a buffer never subsequently read is a *dead store* that optimizing compilers legally remove; this hazard and its persistence across compilers are documented by Yang et al. [11]. The same hazard applies to GPU kernels and is central to our enforcement design (Section 5). A broad GPU side-channel literature studies timing and contention leakage on accelerators [12], [13], but, to our knowledge, no prior work formalizes a per-kernel, attestable local-memory hygiene contract or binds its enforcement to a GPU-CC attestation report. This paper targets that gap.

3. Threat Model and Problem Formalization

3.1 Threat Model

System model. We consider a multi-tenant confidential GPU on which tenants submit kernels that execute inside a GPU-CC protected boundary. Tenants may share the accelerator through intra-VM multiplexing (e.g., MPS/MIG) or through a multi-kernel pipeline within one confidential context. Successive kernels may be mapped to the same physical local-memory banks.

Adversary. The adversary is an honest-but-curious co-tenant who can craft and dispatch arbitrary kernels, optionally aided by a compromised or malicious GPU driver/runtime that schedules kernels and manages buffers. The adversary can allocate local memory and read whatever residue is present in regions it is assigned. The adversary cannot break GPU-CC memory encryption, forge the hardware root of trust, or extract keys from the secure processor.

Assumptions and out-of-scope. We trust the GPU-CC hardware (memory encryption, attestation root of trust) and the CPU TEE. Physical attacks on packaging, interposers, or HBM are out of scope, as are micro-architectural timing/power side channels and speculative-execution attacks beyond direct residue recovery. We initially trust the compiler toolchain that performs the LMHC transformation; Section 6.4 analyzes the consequences of relaxing this assumption and binding compiler identity into the measurement. Our threat model follows the behavior demonstrated by LeftoverLocals, where residual values written by one kernel can remain observable to subsequent kernels on affected GPU platforms when local memory is not explicitly cleared [1].

3.2 Formal Definition: Local Memory Hygiene Contract (LMHC)

Let kernel *K* declare a set of local-memory regions $L(K) = \{l_1, \ldots, l_n\}$. We write |l| for the size of region l in machine words and 0^{|l|} for the all-zero word vector of that length. Let *Conf(K)* denote the set of confidential input parameters of K (e.g., buffers provisioned to the GPU-CC context after attestation).

Definition 1 (Local-State Sensitivity). *A region $l \in L(K)$ is sensitive, written sens(l), iff there exists a data-flow path in K from some confidential input $c \in Conf(K)$ to a write into l. The sensitive set is $S(K) = \{ l \in L(K) : sens(l) \}$.*

Intuitively, l is sensitive if any value transitively derived from a confidential input is ever stored into it. Sensitivity is computed conservatively (over-approximated) so that no leaking region is missed.

Definition 2 (Zeroization Invariant Z). *Let Exit(K) be the set of exit points of K. The zeroization invariant requires that for every sensitive region and every exit, the contents observable by any subsequently scheduled kernel sharing the same physical local memory are zero:*

$\forall l \in S(K), \forall e \in Exit(K) : post_e(l) = 0^{|l|},$ where $post_e(l)$ denotes the externally observable state of l after control leaves K through e.

Definition 3 (Local Memory Hygiene Contract). *An LMHC for kernel K is the triple $C(K) = \langle S(K), Z, M \rangle$, where S(K) is the sensitive set, Z is the zeroization invariant of Definition 2, and M is a compliance witness: a static property of the compiled kernel binary certifying that zeroization of every $l \in S(K)$ dominates every exit in Exit(K) and is not removed by optimization. Kernel K satisfies C(K) iff Z holds under the threat model and M is verifiable by an attesting party.*

The witness M is what makes the contract *attestable*: rather than trusting that scrubbing ran, a verifier can check a measurement that the scrubbing is present and dominating in the shipped binary (Section 6).

3.3 Security Properties

We require three properties of any correct LMHC enforcement.

Confidentiality. No co-tenant kernel can recover any bit of the sensitive local state of a prior kernel: residue reads of S(K) yield only zeros.

Verifiability. The compliance witness M is checkable by a remote verifier through attestation (the compliance bit and kernel hash of Section 6), without trusting the workload operator.

Completeness. Enforcement covers every sensitive region in S(K) and every exit the compiler can statically enumerate—normal return, early returns, and structured fall-through. Asynchronous or abnormal termination is treated as a platform-dependent limitation (Section 7.1).

Theorem 1 (Hygiene Soundness, informal). *If the compiled binary of K places, at every statically reachable exit e ∈ Exit(K), a zeroization of every l ∈ S(K) that (i) post-dominates the last legitimate read of l and (ii) is resistant to dead-store elimination, then under the honest-but-curious co-tenant model no kernel scheduled after K on the same local memory can observe non-zero sensitive residue.*

Proof sketch. By construction, control leaving K passes through some e ∈ Exit(K). At e, each l ∈ S(K) has been overwritten with 0^{|l|}, and because the store post-dominates the last read and is not elided, the write is the final externally observable effect on l. A subsequent kernel mapped to the same banks therefore reads zeros for every word of l, so the recovered value is independent of any confidential input to K. Regions outside S(K) carry no confidential-derived data by Definition 1. ■

4. LMHC Enforcement: A Compiler-Level Approach

4.1 Overview of the Transformation Pipeline

We enforce the LMHC by a source-to-source transformation applied before back-end code generation. The pipeline, depicted in Figure 1, proceeds in five stages: (1) parse the kernel to an intermediate representation and identify confidential input parameters via annotation or provenance; (2) run taint analysis to compute the sensitive set S(K); (3) enumerate the exit points Exit(K) and the last-read (post-dominator) location for each sensitive region; (4) insert cooperative zeroization at each exit for each sensitive region; and (5) harden the inserted code against dead-store elimination with volatile qualification and a memory-fence barrier. The result is recompiled by the unmodified back end.

[Figure 1 placeholder]

Compiler transformation pipeline: source → IR + confidential annotation → taint analysis → exit/post-dominator analysis → scrub insertion → volatile/fence hardening → emit.

Fig. 1. Overview of the LMHC source-to-source enforcement pipeline. [ADD: replace with the actual rendered pipeline diagram]

4.2 Sensitive-Buffer Identification (Taint Analysis)

Confidential inputs are marked at the source level with a `__confidential` qualifier (or inferred from buffers the runtime provisioned into the attested context). Taint is propagated forward to a fixpoint: a value becomes tainted if it is computed from a tainted operand, and a local buffer joins S(K) if any tainted value is written to it. The analysis is intentionally conservative—an over-approximation—so that soundness (Definition 1) is preserved at the cost of occasionally scrubbing a buffer that carried no secret. Algorithm 1 summarizes the computation.

Algorithm 1: *Sensitive-buffer identification.*

```
Input : kernel K, confidential params P
Output: sensitive local set S
  T <- P                 // tainted values
  S <- {}
  repeat
    for each stmt s in K:
      if s defines v from expr e and
         uses(e) intersect T != {}:
        T <- T union {v}
        if target(v) is local buffer b:
          S <- S union {b}
  until T reaches a fixpoint
  return S
```

4.3 Scrubbing-Insertion Algorithm

For each exit and each sensitive buffer, the transformation inserts a block-cooperative zeroing loop in which the threads of the block stride across the buffer, followed by a barrier so that the writes are complete and ordered before control leaves the kernel. Algorithm 2 gives the procedure; the loop is grid-stride within the block to remain correct for arbitrary block sizes.

Algorithm 2: *Exit-point scrub insertion.*

```
Input : kernel K, sensitive set S
  E <- ExitPoints(K)     // returns, fall-through
  for each e in E:
    for each b in S (placed after last read of b):
      emit at e:
        volatile word* vb = (volatile word*) b;
        for (i = tid; i < len(b); i += blockDim.x)
          vb[i] = 0;
      emit barrier:  __threadfence_block();
                     __syncthreads();
  return transformed K
```

4.4 The Dead-Store Elimination Problem and a Volatile-Barrier Solution

A subtlety threatens correctness. Because the scrubbed buffer is, by definition, never read again within K, a naive zeroing loop is a dead store that an optimizing compiler may legally delete—exactly the failure mode documented for CPU secret-wiping code [11]. The transformation must therefore make the stores observable. We combine three measures: (i) access the buffer through a `volatile`-qualified pointer so the stores cannot be elided; (ii) issue a block-scope memory fence (`__threadfence_block()`) to prevent reordering of the zeroing past the kernel boundary; and (iii) place a barrier (`__syncthreads()`) so all threads complete scrubbing before any thread exits. Where a vendor back end is especially aggressive, an inline-assembly store (e.g., a PTX `st.shared`) with a memory clobber provides an opaque sink that the optimizer must respect. Crucially, the scrub is positioned at the post-dominator of the buffer's last legitimate use, so functionality is unaffected.

4.5 Worked Example: LLM Token-Buffer Scrubbing

Consider a kernel that stages attention logits or key/value tiles in local memory. Listing 1 shows the unprotected kernel: the sensitive buffer `buf` is populated from a confidential input and then consumed, but is never cleared. Listing 2 shows the LMHC-enforced version emitted by the transformation, with the volatile, fenced, barrier-synchronized scrub appended at the single exit.

```
// Listing 1: unprotected kernel (residue remains)
__global__ void stage(const float* __confidential in,
                      float* out) {
    __shared__ float buf[256];
    int t = threadIdx.x;
    buf[t] = transform(in[t]);   // tainted write -> buf in S(K)
    __syncthreads();
    out[t] = consume(buf, t);
    // no scrub: buf retains confidential-derived residue
}
```

Listing 1. Unprotected kernel; `buf` is sensitive but left un-scrubbed at exit.

```
// Listing 2: LMHC-enforced kernel (compiler-inserted scrub)
__global__ void stage(const float* __confidential in,
                      float* out) {
    __shared__ float buf[256];
    int t = threadIdx.x;
    buf[t] = transform(in[t]);
    __syncthreads();
    out[t] = consume(buf, t);
    // ---- LMHC scrub (inserted, DSE-resistant) ----
    volatile float* vb = buf;
```

```
    for (int i = t; i < 256; i += blockDim.x)
        vb[i] = 0.0f;
    __threadfence_block();
    __syncthreads();
}
```

Listing 2. Transformed kernel enforcing the zeroization invariant Z at the kernel exit.

5. LMHC Attestation Binding

5.1 The Existing GPU-CC Attestation Report

A GPU-CC attestation report cryptographically conveys the device's identity and the integrity of its firmware and configuration. On Hopper, the GPU measures its firmware during secure boot, signs the measurement together with key-exchange parameters using a device certificate chain, and a verifier compares the result against a reference golden manifest [3], [4]; the AMD Secure Processor issues an analogous signed report for the SEV-SNP CVM [5], and device measurements are carried over SPDM [9]. These reports establish that the platform is genuine and running expected firmware, but they contain no statement about per-kernel memory hygiene.

5.2 Proposed Extension: LMHC Compliance Bit and Kernel Hash

We propose augmenting the attestation evidence with two fields. First, a one-bit `lmhc_enforced` flag, set by the trusted compiler/loader when every kernel admitted to the confidential context carries a verified LMHC witness M. Second, a per-kernel measurement hash, *H(kernel_binary ∥ lmhc_policy)*, accumulated into a dedicated measurement register so that a verifier can match the loaded kernels and policy against a known reference. Listing 3 sketches the extended report fields.

```
// Listing 3: proposed attestation report extension
struct lmhc_attestation_ext {
    uint8_t  lmhc_enforced;     // 1 = all kernels carry verified M
    uint8_t  reserved[3];
    uint8_t  kernel_hash[48];   // SHA-384(kernel_bin || policy)
    uint32_t policy_version;
    uint32_t scrub_region_bytes;
};
```

Listing 3. Candidate fields appended to the signed GPU-CC attestation evidence. [VERIFY: align field layout with the actual vendor/SPDM measurement structure]

5.3 Remote Attestation Flow

Figure 2 depicts the end-to-end flow. A relying party that wishes to send confidential data to a GPU workload (1) requests an attestation report; (2) the GPU/firmware returns a signed report including the standard firmware measurements plus the `lmhc_enforced` bit and kernel hash; (3) the verifier validates the device certificate chain and golden firmware measurement, then checks that the LMHC bit is set and that the kernel hash matches an approved reference; and (4) only upon success does the relying party release secrets (e.g., prompts or model weights) into the context. The hygiene guarantee is thus made a precondition of data provisioning rather than a post-hoc hope.

[Figure 2 placeholder]

Remote attestation flow: relying party → (request) → GPU-CC root of trust → (signed report: firmware measurement + lmhc_enforced bit + kernel hash) → verifier (cert chain + golden RIM + LMHC check) → release secrets on success.

Fig. 2. LMHC-bound remote attestation: hygiene enforcement is verified before confidential data is provisioned. [ADD: replace with the actual rendered attestation-flow diagram]

5.4 Trust-Chain Analysis

The binding shifts a portion of trust onto whoever produces the compliance witness M. In the baseline design, a trusted compiler emits M and a trusted in-TCB loader verifies it before setting the bit; the hardware root of trust

then signs the report. If the compiler is removed from the trusted base, its identity and version must themselves be measured and bound into the attestation so a verifier can reject witnesses from an untrusted toolchain [VERIFY: confirm whether current GPU-CC measured-boot chains can incorporate a compiler/build-provenance measurement]. A stronger but harder alternative, which we leave to future work, is an in-GPU verifier that re-derives the dominator property directly from the loaded binary, eliminating compiler trust entirely. In all variants, the cryptographic chain—device key → signed measurement → verifier policy—remains rooted in the existing GPU-CC hardware.

6. Proof-of-Concept Evaluation

6.1 Setup and Assumptions

We describe a conceptual prototype rather than a full implementation; the kernels below are CUDA-like pseudocode that illustrate the leakage and its remediation. [ADD: replace this section's qualitative claims and the overhead table with measurements from a real implementation on target hardware] Consistent with the behavior demonstrated in LeftoverLocals, we assume that local-memory residue can remain observable across kernel launches on affected GPU platforms when local memory is not explicitly scrubbed [1].

6.2 Demonstrating the Leakage

Listing 4 shows a writer kernel that places a recognizable secret pattern in local memory and exits without scrubbing, and a listener kernel that reads the uninitialized region and exfiltrates it. In the vulnerable configuration, the listener recovers the writer's pattern with high fidelity (estimated > 90% of words for buffers that are fully reused) [ADD: replace with measured recovery rate].

```
// Listing 4: leakage demonstration
__global__ void writer(const float* __confidential in) {
    __shared__ float buf[256];
    int t = threadIdx.x;
    buf[t] = secret_pattern(in[t]); // sensitive
    __syncthreads();
    use(buf);            // no scrub at exit
}

__global__ void listener(float* out) {
    __shared__ float buf[256];      // same banks, reused
    int t = threadIdx.x;
    out[t] = buf[t];     // reads prior residue
}
```

Listing 4. Writer leaves residue; listener recovers it across the kernel boundary.

6.3 Demonstrating the Fix

Applying the LMHC transformation to the writer (Listing 5) appends the DSE-resistant scrub at exit. Re-running the listener now yields an all-zero buffer, so no confidential-derived value is recoverable, consistent with Theorem 1.

```
// Listing 5: writer after LMHC enforcement
__global__ void writer(const float* __confidential in) {
    __shared__ float buf[256];
    int t = threadIdx.x;
    buf[t] = secret_pattern(in[t]);
    __syncthreads();
    use(buf);
    // ---- inserted scrub ----
    volatile float* vb = buf;
    for (int i = t; i < 256; i += blockDim.x)
        vb[i] = 0.0f;
    __threadfence_block();
    __syncthreads();
}
```

Listing 5. After enforcement, the listener of Listing 4 reads only zeros.

6.4 Overhead Estimation

Let the sensitive footprint be B bytes cleared cooperatively by a block of P threads issuing w-byte stores at on-chip store throughput τ (bytes/cycle). The added latency is approximately $B / \min(P \cdot w \cdot f, \tau)$ cycles per exit, where f is the issue rate; because local memory is on-chip and the clear is fully parallel across the block, this term is small relative to a kernel's own compute and global-memory traffic. Table I gives order-of-magnitude estimates for representative kernel classes. All figures in Table I are theoretical estimates and must be replaced with measured overheads [ADD: real benchmark results] The qualitative conclusion is that for memory-bound and GEMM-dominated LLM kernels, where each byte of local memory is amortized over substantial arithmetic, the relative cost of a single zeroing pass at exit is modest.

Kernel class	Local mem used (est.)	Est. scrub overhead
Memory-bound (elementwise)	up to 48 KB	< 1%
Compute-bound GEMM (large tiles)	32–64 KB	1–3%
LLM inference (attention, small tiles)	48–100 KB	2–5%

TABLE I. *Estimated per-exit scrubbing overhead by kernel class* [ADD: replace estimates with measured results]

7. Discussion

7.1 Limitations and Scope

Our enforcement covers the on-chip local-memory scratchpad and the exits the compiler can statically enumerate. Three limitations follow. First, asynchronous or abnormal kernel termination may bypass structured exits; closing this requires platform support for scrub-on-teardown [VERIFY: determine whether target runtimes expose a hook for abnormal-termination scrubbing]. Second, per-thread register spills that the toolchain places in CUDA-style local (global-backed) memory are outside our current scope; such spills can also carry secrets and warrant a complementary mechanism. Third, the evaluation is conceptual, and the quantitative claims are estimates pending implementation.

7.2 Generalizability to AMD, Apple, and Qualcomm GPUs

The LMHC and the source-to-source transformation are vendor-agnostic at the language level and apply to CUDA, HIP, OpenCL, and Metal kernels; only the barrier and fence intrinsics, and the inline-store fallback, are vendor-specific. The attestation binding, however, presupposes a device attestation facility. NVIDIA Hopper provides one [3], and AMD offers CPU-side and trusted-I/O attestation [5], [8], but Apple and Qualcomm GPUs do not expose a comparable public GPU attestation interface at present, which limits the binding on those platforms [VERIFY: confirm current attestation support on Apple/Qualcomm GPUs]. On such devices the compiler enforcement still delivers the confidentiality property, while verifiability awaits hardware support.

7.3 Future Work

Three directions stand out. First, hardware-assisted scrubbing: a “scrub-on-exit” control bit that zeroes a block's local memory in hardware would remove the dead-store hazard and reduce overhead. Second, formal verification: mechanically proving that the transformation establishes Z (Definition 2) for all exits would turn the informal Theorem 1 into a machine-checked guarantee. Third, in-GPU witness re-verification and coverage of register spills, together with integration of the transformation into NVCC and ROCm so that LMHC enforcement becomes a default compiler pass rather than an external tool.

8. Conclusion

LeftoverLocals showed that unscrubbed GPU local memory leaks LLM data across tenants, and GPU Confidential Computing, while essential, scrubs only at VM-reassignment granularity—leaving inter-kernel local-memory hygiene unaddressed and unattested. We formalized this missing guarantee as the Local Memory Hygiene Contract, gave a compiler-level enforcement that inserts optimization-resistant scrubbing at kernel exits, and

proposed an attestation binding that makes hygiene a verifiable precondition for provisioning confidential data. A conceptual proof of concept and a theoretical overhead analysis suggest the approach is both effective and affordable. We offer the LMHC as a foundation for standardizing per-kernel memory hygiene in confidential GPU computing, and we encourage vendors and toolchain maintainers to adopt and measure it.

References

[1] Sorensen, T., & Khlaaf, H. (2024). LeftoverLocals: Listening to LLM responses through leaked GPU local memory. *arXiv preprint arXiv:2401.16603*.

[2] National Vulnerability Database. (2024). CVE-2023-4969 detail. National Institute of Standards and Technology. https://nvd.nist.gov/vuln/detail/CVE-2023-4969

[3] NVIDIA. (2023). Confidential Compute on NVIDIA Hopper H100. NVIDIA White Paper WP-11459-001. https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/HCC-Whitepaper-v1.0.pdf

[4] Dhanuskodi, G., Guha, S., Krishnan, V., Manjunatha, A., Nertney, R., O'Connor, M., & Rogers, P. (2023). Creating the first confidential gpus. *Communications of the ACM*, *67*(1), 60-67.

[5] Advanced Micro Devices. (2020). AMD SEV-SNP: Strengthening VM isolation with integrity protection and more (Document 70366). https://docs.amd.com/v/u/en-US/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more

[6] Kaplan, D. (2016). {AMD} x86 Memory Encryption Technologies.

[7] Intel, "Intel Trust Domain Extensions (Intel TDX) White Paper," 2023.

[8] AMD, "AMD SEV-TIO: Trusted I/O for Secure Encrypted Virtualization," White Paper, 2023.

[9] Distributed Management Task Force. (2022). Security Protocol and Data Model (SPDM) Specification (Version 1.2.1, DSP0274). https://www.dmtf.org/dsp/DSP0274

[10] Trusted Computing Group. (2024). TCG Reference Integrity Manifest (RIM) Information Model (Version 1.1 Revision 1.0). https://trustedcomputinggroup.org/resource/tcg-reference-integrity-manifest-rim-information-model/

[11] Yang, Z., Johannesmeyer, B., Olesen, A. T., Lerner, S., & Levchenko, K. (2017). Dead store elimination (still) considered harmful. In *26th USENIX Security Symposium (USENIX Security 17)* (pp. 1025-1040).

[12] Naghibijouybari, H., Neupane, A., Qian, Z., & Abu-Ghazaleh, N. (2018, October). Rendered insecure: Gpu side channel attacks are practical. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security* (pp. 2139-2153).

[13] Jiang, Z. H., Fei, Y., & Kaeli, D. (2016, March). A complete key recovery timing attack on a GPU. In *2016 IEEE International symposium on high performance computer architecture (HPCA)* (pp. 394-405). IEEE.

[14] Wang, Q., & Oswald, D. (2026). Confidential computing on heterogeneous cpu-gpu systems: Survey and future directions. *ACM Computing Surveys*.

[15] Zhu, J., Yin, H., Deng, P., Almeida, A., & Zhou, S. (2024). Confidential computing on NVIDIA Hopper GPUs: a performance benchmark study. *arXiv preprint arXiv:2409.03992*.

[16] Misono, M., Stavrakakis, D., Santos, N., & Bhatotia, P. (2024). Confidential vms explained: An empirical analysis of amd sev-snp and intel tdx. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, *8*(3), 1-42.

[17] NVIDIA, "NVIDIA H100 Tensor Core GPU Architecture," White Paper, 2022.

[18] Khronos Group. (2023). The OpenCL Specification (Version 3.0). https://registry.khronos.org/OpenCL/specs/

[19] NVIDIA. (2024). CUDA C++ Programming Guide. NVIDIA Corporation. https://docs.nvidia.com/cuda/

[20] Volos, S., Vaswani, K., & Bruno, R. (2018). Graviton: Trusted execution environments on {GPUs}. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)* (pp. 681-696).

[21] Jang, I., Tang, A., Kim, T., Sethumadhavan, S., & Huh, J. (2019, April). Heterogeneous isolated execution for commodity gpus. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (pp. 455-468).

[22] Cremers, C., Dax, A., & Naska, A. (2023). Formal analysis of {SPDM}: Security protocol and data model version 1.2. In *32nd USENIX Security Symposium (USENIX Security 23)* (pp. 6611-6628).